Computer Architecture - Staff Graded Assignment

Student ID: 2025EB01187

Name: Amman Jakhar

Reference worked solutions. All calculations shown. (Study aid - rewrite in your own hand for submission.)

Q1. Performance and Amdahl's Law

Mix: 50% vector, 30% floating-point, 20% control. Amdahl's Law: Speedup = 1 / [(1 - f) + f/s].

(a) Individual upgrades

```
Upgrade A (vector, f=0.50, s=4):
new time = 0.50 + 0.50/4 = 0.625 -> Speedup = 1/0.625 = 1.60x
Upgrade B (FP, f=0.30, s=6):
new time = 0.70 + 0.30/6 = 0.75 -> Speedup = 1/0.75 = 1.333x
```

Upgrade A gives the greater speedup (1.60x > 1.333x) because it accelerates the larger fraction of execution time.

(b) Both upgrades together

```
new time = 0.20 + 0.50/4 + 0.30/6 = 0.20 + 0.125 + 0.05 = 0.375
Overall speedup = 1/0.375 = 2.667x
```

Q2. MIPS Instruction Formats

Register numbers: $t0=8, $t1=9, $s0=16, $s1=17, $sp=29, $ra=31.

sub $t1, $s0, $s1 -> R-type

```
op=000000 | rs=$s0=16 (10000) | rt=$s1=17 (10001)
rd=$t1=9 (01001) | shamt=00000 | funct=0x22 (100010)
```

Role: ALU subtract; funct selects subtraction; result written to rd ($t1).

ori $t0, $t1, 0xFF00 -> I-type

```
op=0x0D (001101) | rs=$t1=9 (01001) | rt=$t0=8 (01000)
immediate=0xFF00 (1111111100000000) [zero-extended]
```

Role: bitwise OR of $t1 with the zero-extended immediate; result -> $t0.

sw $s0, 16($sp) -> I-type

```
op=0x2B (101011) | rs(base)=$sp=29 (11101) | rt(data)=$s0=16 (10000)
offset/immediate=16 (0000000000010000) [sign-extended]
```

Role: store word; effective address = $sp + 16; value in $s0 written to memory.

jal ProcedureLabel -> J-type

```
op=0x03 (000011) | 26-bit address = target word address (target >> 2)
```

Role: jump and link; saves return address (PC+4) in $ra, then jumps to the target.

Q3. Address Decoder (16 KB)

(a) Byte-addressable

```
16 KB = 16 x 1024 = 16384 bytes = 2^14
```

```
Input lines m = 14
Output lines n = 2^14 = 16384 (one per byte location)
```

(b) Word-addressable (1 word = 32 bits = 4 bytes)

```
16384 bytes / 4 = 4096 words = 2^12
Input lines m = 12 ; Output lines n = 2^12 = 4096
```

Architectural difference: each address now selects a 4-byte word instead of a byte, so the addressing granularity is coarser - 2 fewer address bits and only a quarter as many output lines.

Q4. IEEE-754 Single Precision

```
Pattern: 0 10000010 01100000000000000000000
```

(a) Field identification

```
Sign S = 0 (positive)
Biased exponent E = 10000010
Fraction F = 01100000000000000000000
```

(b) Unbiased exponent and normalized form

```
E = 10000010 = 130 ; bias = 127
true exponent = 130 - 127 = +3
implicit leading bit = 1 -> significand = 1.011
Normalized: 1.011 x 2^3
```

(c) Decimal value

```
1.011(2) = 1 + 0.25 + 0.125 = 1.375
value = 1.375 x 2^3 = 1.375 x 8 = +11.0
```

Q5. Multi-Cycle MIPS Cycle Counts

(a) Cycles per instruction

```
lw $t0, 4($s0) : 5 cycles (IF ID EX MEM WB)
add $s1, $t0, $t2 : 4 cycles (IF ID EX WB)
sw $t0, 4($s0) : 4 cycles (IF ID EX MEM)
bne $t0, $t1, Target : 3 cycles (IF ID EX) [taken]
```

(b) Stage-by-stage trace

```
lw : IF=fetch | ID=decode,read regs | EX=compute $s0+4 |
MEM=read memory | WB=write loaded value to $t0 (5 cycles)
sw : IF=fetch | ID=decode,read regs | EX=compute $s0+4 |
MEM=write $t0 to memory (4 cycles)
```

Difference: lw must write the fetched data back to a register (WB stage), while sw writes to memory and produces no register result, so it omits WB. Hence lw = 5, sw = 4.

Q6. Average CPI

(a) Baseline CPI

```
CPI = 0.40(1) + 0.30(4) + 0.15(3) + 0.15(2)
= 0.40 + 1.20 + 0.45 + 0.30 = 2.35
```

(b) Optimized CPI and net speedup

```
New CPI = 0.40(1) + 0.30(2) + 0.15(2) + 0.15(2)
= 0.40 + 0.60 + 0.30 + 0.30 = 1.60
```

```
Clock period x 1.10. Time = IC x CPI x period.
Speedup = (2.35 x T) / (1.60 x 1.10 x T) = 2.35 / 1.76 = 1.335x
```

Q7. Single-Cycle vs Multi-Cycle

Program: 20 ALU + 10 Load + 10 Store = 40 instructions.

(a) Single-cycle datapath

```
Clock period = longest path = Load = 45 ns
Total time = 40 x 45 ns = 1800 ns
```

(b) Multi-cycle datapath (10 ns/stage)

```
Cycles = 20(4) + 10(5) + 10(4) = 80 + 50 + 40 = 170
Total time = 170 x 10 ns = 1700 ns
Faster design = multi-cycle. Speedup = 1800/1700 = 1.06x
```

Q8. Pipeline Data Hazards (lw -> add -> sub)

Assumption: a value written in WB can be read in ID in the same cycle.

(a) RAW hazards

```
lw writes $t0; add reads $t0 -> RAW (lw -> add)
lw writes $t0; sub reads $t0 -> RAW (lw -> sub)
```

(b) Without forwarding: 2 stalls

```
Cycle: 1 2 3 4 5 6 7 8
lw : IF ID EX MEM WB
add: IF -- -- ID EX MEM WB
sub: -- -- IF ID EX MEM WB
```

$t0 is produced in WB (cycle 5) and read in ID the same cycle, so add's ID slides to cycle 5 -> 2 bubbles.

(c) With full forwarding: 1 stall

```
Cycle: 1 2 3 4 5 6 7
lw : IF ID EX MEM WB
add: IF ID -- EX MEM WB
sub: IF -- ID EX MEM WB
```

Load data is ready after MEM (cycle 4) but add needs it at the start of EX, so one load-use bubble is unavoidable; then forward MEM->EX. sub is far enough to take the forwarded value with no extra stall.

Q9. Direct-Mapped Cache

32-bit address, byte-addressable, 16 KB cache, 32-byte blocks.

(a) Number of lines

```
Lines = 16 KB / 32 B = 16384 / 32 = 512 lines
```

(b) Field widths

```
Offset = log2(32) = 5 bits
Index = log2(512) = 9 bits
Tag = 32 - 9 - 5 = 18 bits
```

(c) Address 0x0002A45C

```
binary = 0000 0000 0000 0010 1010 0100 0101 1100
```

```
Offset = low 5 bits = 11100 = 28 (0x1C)
Index = next 9 bits = 100100010 = 290 (0x122)
Tag = top 18 bits = 0000...1010 = 10 (0xA)
```

Q10. Single-Cycle Control Signals

(a) Control signals for lw

```
RegDst = 0 (write register is rt)
ALUSrc = 1 (ALU second operand = sign-extended immediate)
MemtoReg = 1 (reg write data comes from memory)
RegWrite = 1 (write result to register file)
MemRead = 1 (read data memory)
MemWrite = 0
Branch = 0
```

(b) Why RegWrite = 1 during sw corrupts state

sw produces no register result; it only writes a value to memory. If RegWrite were 1, the register file would be overwritten with a meaningless value (the computed effective address on the write-data path), clobbering a register the program relies on and corrupting architectural state. Therefore sw must keep RegWrite = 0.